

React 19 y Hono

Interfaces declarativas y servicios sobre Web Standards

Desarrollo de Software IX · Código 1493 · Módulo III · Preparación Parcial #3

Propósito del material

Desarrollar la teoría del frontend declarativo con React —incluyendo la evolución que introduce React 19— y del backend sobre Web Standards con Hono, junto con los criterios de ingeniería para evaluar ambas tecnologías como solución.

1. Panorama

Una aplicación Full Stack moderna se sostiene sobre dos decisiones de arquitectura: cómo se construye la interfaz y dónde —y sobre qué contrato— se ejecutan los servicios. Este material desarrolla la teoría detrás de ambas: del lado del cliente, el paradigma declarativo de React y su evolución más reciente (React 19); del lado del servidor, el regreso a los estándares de la plataforma web que representa Hono, un framework diseñado para ejecutarse en cualquier runtime de JavaScript, incluido el edge.

El recorrido complementa dos capítulos del Módulo III: el Capítulo 7, que presenta React (componentes, hooks y formularios), y el Capítulo 5, que estudia la seguridad de APIs con JSON Web Token. Sirve como material de preparación para el tercer parcial del curso.

Idea central

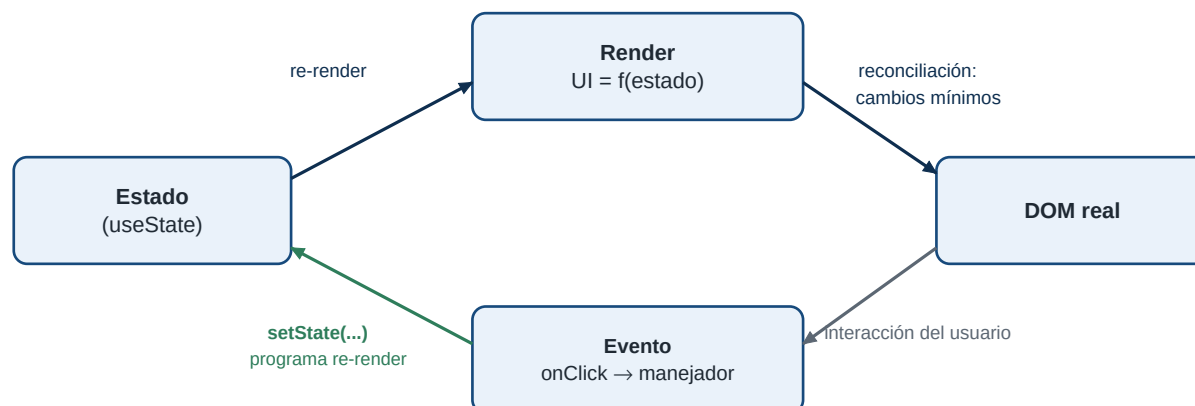
React y Hono comparten una misma filosofía desde extremos opuestos de la aplicación: describir QUÉ debe ocurrir y delegar el CÓMO — React lo hace con la interfaz (la UI es una función del estado), Hono con la plataforma (el servicio es una función de Request a Response).

2. El paradigma declarativo de React

2.1 La interfaz como función del estado

En la programación imperativa de interfaces (el modelo del DOM clásico y jQuery), el programador manipula directamente los nodos: busca un elemento, le cambia el texto, agrega o quita clases. A medida que la aplicación crece, el número de transiciones posibles entre estados visuales crece de forma combinatoria, y cada transición es código que alguien debe escribir y mantener.

React invierte el modelo: el programador describe cómo se ve la interfaz PARA un estado dado — $UI = f(\text{estado})$ — y la librería se encarga de las transiciones. Cuando el estado cambia, React vuelve a ejecutar la función del componente y obtiene la nueva descripción. El programador nunca escribe la mutación del DOM; describe el resultado final.



2.2 JSX: describir la interfaz dentro de JavaScript

JSX es la sintaxis con la que se escriben esas descripciones: una extensión de JavaScript que permite expresar estructuras similares a HTML dentro del código. No es HTML (se usa `className` en lugar de `class` y las expresiones se interpolan entre llaves), no es un motor de plantillas que se procesa en el servidor, y no reemplaza a CSS: cada fragmento JSX se compila a llamadas de función de JavaScript que construyen elementos.

```
function Saludo({ nombre }) {
  return <h1 className="titulo">Hola, {nombre}</h1>
}
```

2.3 Virtual DOM y reconciliación

Si en cada cambio de estado React reconstruyera el DOM completo, el modelo declarativo sería impagablemente lento: el DOM real es costoso de modificar. La solución es el Virtual DOM: una representación ligera en memoria del árbol de elementos. En cada render React produce un árbol nuevo, lo compara con el anterior (proceso llamado reconciliación o diffing) y calcula el conjunto mínimo de operaciones para llevar el DOM real del estado viejo al nuevo. Por eso un cambio de estado no re-renderiza toda la página ni requiere consultar nada al servidor: es una comparación local entre dos descripciones.

La reconciliación explica también por qué las listas necesitan la prop `key`: cuando React compara dos versiones de una lista, la `key` es la identidad estable de cada elemento. Con ella puede saber si un elemento se movió, se agregó o se eliminó; sin ella solo puede comparar por posición, lo que produce actualizaciones incorrectas o innecesarias cuando la lista se reordena.

```
{usuarios.map((u) => (
  <li key={u.id}>{u.nombre}</li>    // key = identidad estable
))}
```

2.4 Componentes, props y flujo unidireccional

El componente es la unidad de composición: una función que recibe datos y devuelve una descripción de interfaz. Los datos de entrada — los props — son de SOLO LECTURA. Esta inmutabilidad no es un capricho: garantiza que un componente sea predecible (con los mismos props produce la misma salida) y que los datos fluyan en una sola dirección, de padres a hijos. Cuando un componente necesita datos que cambian en el tiempo, eso ya no es un prop: es estado.

Los eventos cierran el ciclo: el usuario interactúa, el manejador actualiza el estado, y el nuevo estado produce un nuevo render. En JSX el manejador se asocia pasando la REFERENCIA de la función — `onClick={manejar}` — no su invocación (`onClick={manejar()}`) la ejecutaría en cada render, no al hacer clic).

3. Estado, hooks y ciclo de vida

3.1 Por qué existen los hooks

En el React original, solo los componentes de clase podían tener estado y ciclo de vida, y reutilizar esa lógica entre componentes exigía patrones indirectos. Los hooks (2019) resolvieron ambos problemas: dan estado y efectos a los componentes de función, y permiten extraer lógica reutilizable en custom hooks (funciones que combinan hooks existentes).

React identifica cada hook por el ORDEN en que se llama durante el render. De ahí derivan las reglas — no son convenciones de estilo, son requisitos del modelo:

- Se llaman solo en el nivel superior del componente: un hook dentro de un condicional o un bucle cambiaría el orden de llamadas entre renders y rompería la correspondencia.
- Se llaman solo desde componentes de función o custom hooks: fuera de un render no existe la estructura que les da significado.
- Sus nombres comienzan con use: permite a las herramientas verificar automáticamente las dos reglas anteriores.

3.2 useState: el estado como instantánea

useState devuelve el valor actual y una función para actualizarlo. Llamar al setter no muta la variable de inmediato: registra el nuevo valor y PROGRAMA un re-render. Durante un mismo render, el valor del estado es una instantánea congelada; el valor nuevo solo existe en el render siguiente. Este modelo hace el flujo de datos trazable: cada render es una foto consistente del estado.

```
const [count, setCount] = useState(0)
setCount(count + 1) // registra el cambio y programa un re-render
```

3.3 useEffect: sincronizar con el exterior

El render de React debe ser puro: calcular la descripción de la UI y nada más. Todo lo que toca el mundo exterior — peticiones HTTP, suscripciones, timers, el título del documento — es un efecto, y vive en useEffect, que se ejecuta DESPUÉS del render. El arreglo de dependencias declara con qué valores está sincronizado el efecto:

Dependencias	Cuándo se ejecuta el efecto
useEffect(fn)	Después de cada render
useEffect(fn, [])	Una sola vez, después del primer render (montaje)
useEffect(fn, [a, b])	Tras el primer render y cada vez que a o b cambien

3.4 Formularios: el principio de la fuente única de verdad

Un formulario plantea una pregunta de diseño: ¿quién es el dueño del valor del input, el DOM o React? En un componente NO controlado, el DOM guarda el valor y React lo lee cuando lo necesita. En un componente CONTROLADO, el valor vive en el estado de React y el input lo refleja: cada tecla dispara onChange, que actualiza el estado, que re-renderiza el input. El estado se convierte en la fuente única de verdad, lo que habilita validación inmediata, campos dependientes y envíos deshabilitados mientras los datos sean inválidos.

```
const [email, setEmail] = useState('')

<input value={email} onChange={(e) => setEmail(e.target.value)} />
```

4. React 19: la evolución del modelo

Las versiones recientes de React atacan un mismo problema de fondo: el código asíncrono repetitivo. Cargar datos, enviar formularios y reflejar operaciones pendientes obligaba a cada equipo a reinventar los mismos estados manuales (cargando, error, éxito). React 19 — versión estable actual, 19.2 — incorpora ese patrón al núcleo de la librería.

API	Concepto que incorpora
use()	Leer recursos (una Promise o un Context) durante el render; al no ser un hook clásico, puede usarse dentro de condicionales.
Actions + useActionState	El ciclo completo de una mutación — envío, estado pendiente, error — gestionado por React en lugar de a mano.
useOptimistic	Interfaz optimista: asumir el resultado esperado mientras la operación se confirma.
ref como prop	Simplificación del API: los componentes de función reciben ref como cualquier otro prop, sin forwardRef.
React Compiler	Optimización automática en tiempo de compilación; la memorización deja de ser responsabilidad del programador.

4.1 use(): recursos como valores

use() generaliza una idea: una Promise o un Context son recursos que el render puede leer. A diferencia de los hooks, use() puede llamarse dentro de condicionales, porque React no lo rastrea por orden de llamada sino por el recurso que recibe. Combinado con Suspense, leer una Promise suspende el componente hasta que el dato esté disponible:

```
import { use } from 'react'

function Perfil({ usuarioPromise, conTema }) {
  const usuario = use(usuarioPromise) // lee una Promise
  if (conTema) {
    const tema = use(TemaContext) // legal dentro de un if
  }
  return <h2>{usuario.nombre}</h2>
}
```

4.2 Actions y useActionState: las mutaciones como ciudadanos de primera clase

Una Action es una función asíncrona conectada a un formulario. React ejecuta la Action al enviarlo y expone su ciclo de vida: useActionState entrega el último resultado, la acción que se pasa al form y un booleano de operación pendiente. Los estados que antes se modelaban a mano con varios useState pasan a ser parte del contrato del framework:

```
const [error, submitAction, isPending] = useActionState(
  async (prev, formData) => {
    const err = await guardar(formData.get('nombre'))
    return err ?? null
  },
  null
)

<form action={submitAction}>
  <input name="nombre" />
  <button disabled={isPending}>Guardar</button>
</form>
```

4.3 useOptimistic: la latencia como problema de diseño

Toda operación remota tarda. La interfaz pesimista espera la confirmación del servidor para mostrar el cambio; la optimista lo muestra de inmediato — el mensaje recién enviado aparece en el chat al instante — y lo revierte solo si la operación falla. `useOptimistic` implementa este patrón: mantiene una versión optimista del estado mientras la `Action` está en curso y vuelve sola al estado real al terminar. No guarda relación con optimizar imágenes ni el tamaño del bundle: optimiza la latencia PERCIBIDA.

4.4 ref como prop

`ref` es la vía de escape al mundo imperativo: una referencia directa a un nodo del DOM (para enfocar un input, medir un elemento). Hasta React 18, pasar un `ref` a través de un componente de función exigía envolverlo en `forwardRef`. React 19 elimina esa ceremonia: `ref` llega como un prop más.

```
function MiInput({ ref, ...props }) {
  return <input ref={ref} {...props} />
}
```

4.5 React Compiler: optimización sin intervención

El costo del modelo declarativo es el re-render: cuando un componente se renderiza, sus hijos también, aunque sus props no hayan cambiado. La mitigación clásica era manual — `useMemo`, `useCallback`, `React.memo` — y propensa a olvidos y exceso. El React Compiler analiza el código en tiempo de compilación y aplica esa memorización automáticamente: el programa conserva su forma declarativa y la optimización deja de ser responsabilidad del autor. No convierte React en una app nativa, no elimina el Virtual DOM, no compila a WebAssembly: memoriza lo que antes se memorizaba a mano.

4.6 Integración con APIs: los tres estados de la interfaz

Consumir un servicio remoto introduce incertidumbre, y la teoría de interfaces la modela con tres estados que toda pantalla conectada debe representar: **CARGA** mientras se espera la respuesta, **ERROR** si la petición falla, y **DATOS** cuando llega el resultado. El instrumento concreto puede variar — `fetch` dentro de `useEffect`, `use()` con `Suspense`, una `Action` — pero los tres estados son el contrato entre la interfaz y la red:

```
if (cargando) return <Spinner />
if (error) return <Error mensaje={error.message} />
return <Lista items={datos} />
```

5. Hono: el backend sobre Web Standards

5.1 Contexto: de las APIs propias a los estándares

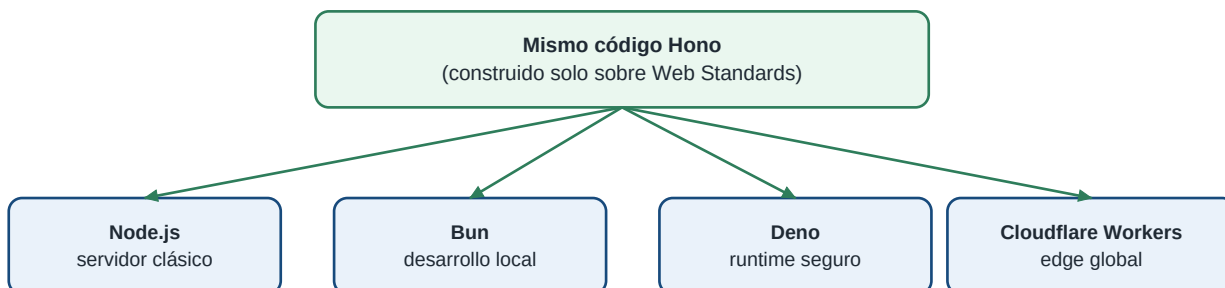
Express (2010) se construyó sobre las primitivas propias de Node.js: los objetos `req` y `res` no existen fuera de ese runtime. Durante una década eso no importó, porque Node era el único lugar donde corría JavaScript de servidor. El panorama cambió con la plataforma web estándar: la especificación `Fetch` definió `Request` y `Response` como objetos universales, y aparecieron nuevos runtimes — `Cloudflare Workers`, `Deno`, `Bun` — que los implementan directamente.

Hono pertenece a esta segunda generación: está construido EXCLUSIVAMENTE sobre Web Standards. La consecuencia es la portabilidad — el mismo código corre en `Workers`, `Bun`, `Deno`, `Node.js` o `AWS Lambda` sin modificación — y la ligereza: su preset mínimo pesa menos de 15 kB, pensado para entornos donde cada milisegundo de arranque cuenta.

5.2 Edge computing: por qué importa dónde corre el código

El modelo tradicional concentra el servidor en una región: un usuario lejano paga la distancia en cada petición (latencia de red) y el servidor permanece encendido aunque no haya tráfico. El edge computing distribuye la ejecución entre decenas de puntos de presencia: cada petición se atiende cerca del usuario, y el código arranca bajo demanda — lo que exige tiempos de arranque (cold start) mínimos. Un framework apto para el edge debe ser pequeño, arrancar rápido y no depender de APIs exclusivas de un runtime. Express no cumple la tercera condición; Hono fue diseñado para las tres.

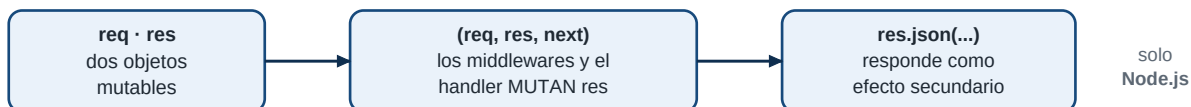
Aspecto	Express	Hono
Base	API propia de Node.js (req, res)	Web Standards (Request/Response) y Context (c)
Runtimes	Node.js	Cloudflare Workers, Bun, Deno, Node.js, AWS Lambda...
Middlewares	Ecosistema externo (express-jwt, cors...)	Integrados: hono/jwt, CORS, logger, etag...
Tipado	Tipos externos (@types/express)	TypeScript de primera clase
Manejo de errores	Middleware de 4 argumentos	HTTPException + app.onError centralizado
Peso y arranque	Pensado para servidor persistente	Ultraligero, pensado para arranque bajo demanda



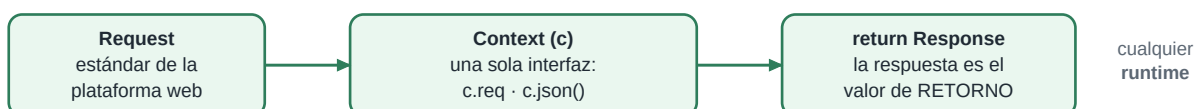
5.3 El objeto Context: rutas como funciones puras

Donde Express divide la petición y la respuesta en dos objetos mutables (req, res), Hono entrega un único Context (c): la petición se lee de c.req y la respuesta es el VALOR DE RETORNO del handler. Conceptualmente, cada ruta es una función de Request a Response — el mismo principio declarativo de React, aplicado al servidor:

Express (API propia de Node.js)



Hono (Web Standards)



```
import { Hono } from 'hono'
const app = new Hono()

app.get('/api/users', (c) => c.json(users))

app.post('/api/users', async (c) => {
  const body = await c.req.json()
  return c.json(crearUsuario(body), 201)
})
```

5.4 Middlewares: la cadena de responsabilidad

Un middleware intercepta la petición antes (y después) del handler: autenticación, registro, CORS, compresión. Hono implementa el patrón de cadena de responsabilidad con funciones `async`: cada eslabón decide si continúa la cadena llamando a `await next()`. Que `next` se espere con `await` — a diferencia de Express — permite ejecutar lógica también DESPUÉS de que el resto de la cadena respondió (medir tiempos, transformar la respuesta):

```
app.use('/api/*', async (c, next) => {
  const inicio = Date.now()
  await next() // continúa la cadena
  console.log(Date.now() - inicio) // se ejecuta de regreso
})
```

5.5 Errores como excepciones del dominio HTTP

En lugar de esparcir `try/catch` por cada ruta, Hono modela los errores con una excepción tipada — `HTTPException`, que transporta el código de estado — y un punto único de captura: `app.onError`. La ruta expresa la condición de error donde la detecta; la traducción a respuesta HTTP vive centralizada:

```
import { HTTPException } from 'hono/http-exception'

app.get('/api/users/:id', (c) => {
  const user = buscarUsuario(c.req.param('id'))
  if (!user) throw new HTTPException(404, { message: 'No existe' })
  return c.json(user)
})

app.onError((err, c) => {
  if (err instanceof HTTPException) return err.getResponse()
  return c.json({ error: err.message }, 500)
})
```

5.6 Seguridad: JWT sin dependencias externas

La teoría del JSON Web Token es la del Capítulo 5: un token firmado (`header.payload.signature`) que el servidor emite al autenticar y verifica en cada petición, sin guardar sesión en memoria — autenticación sin estado, ideal para servicios distribuidos. La diferencia práctica en Hono es que el verificador es un middleware INTEGRADO (`hono/jwt`) que se aplica a las rutas protegidas; los claims del token verificado quedan disponibles en el Context:


```
import { jwt } from 'hono/jwt'

app.use('/api/private/*', jwt({ secret: process.env.JWT_SECRET }))

app.get('/api/private/perfil', (c) => {
  const payload = c.get('jwtPayload')
  return c.json({ usuario: payload.sub })
})
```

6. Evaluar tecnologías: la viabilidad técnica

Elegir un framework es una decisión de ingeniería, no de moda. El Módulo I del curso lo formula como determinar la viabilidad técnica: justificar la selección de tecnologías a partir de los requerimientos. El método es siempre el mismo — identificar los requisitos dominantes del escenario, contrastarlos con las capacidades demostrables de la tecnología, y pesar honestamente los trade-offs. Una recomendación sin limitaciones declaradas es publicidad, no análisis.

6.1 React como solución de frontend

React es candidato natural cuando la interfaz es rica en estado: datos que cambian en el tiempo, formularios complejos, múltiples vistas que comparten lógica. Sus capacidades responden a requisitos concretos:

Requisito del escenario	Capacidad de React que lo atiende
Datos que cambian en tiempo real	Modelo declarativo + reconciliación: la UI se actualiza sola al cambiar el estado, tocando solo lo necesario.
Formularios complejos con validación	Componentes controlados; Actions y useActionState gestionan envío, pendiente y error.
Percepción de fluidez ante la latencia	useOptimistic y Suspense: interfaz optimista y carga progresiva.
Muchas vistas con lógica compartida	Composición de componentes y custom hooks reutilizables.
Rendimiento sostenible al crecer	React Compiler: memorización automática sin esfuerzo del equipo.
Velocidad de desarrollo	Ecosistema maduro (enrutadores, TanStack Query, librerías de gráficas) y comunidad masiva.

Limitaciones a declarar: React es una librería de interfaz, no un framework completo — enrutamiento, datos y build se resuelven con piezas adicionales; el modelo de hooks tiene curva de aprendizaje real; y para sitios mayormente estáticos su peso puede no justificarse.

6.2 Hono frente a Express como solución de backend

La pregunta decisiva es DÓNDE correrá el servicio. Si el requisito es latencia baja para usuarios geográficamente dispersos, el análisis apunta al edge — y en el edge Express no puede correr, porque depende de Node.js. Hono, sobre Web Standards, corre allí y también en Node o Bun durante el desarrollo local: el mismo código en ambos mundos. A eso se suman los middlewares integrados (JWT, CORS, logger reducen dependencias externas) y el tipado TypeScript de primera clase.

Limitaciones a declarar: el ecosistema y la comunidad de Express son mucho mayores (15+ años de middlewares, documentación y respuestas); las librerías ligadas a APIs propias de Node.js pueden no funcionar en runtimes de edge; y un equipo consolidado en Express asume un costo de migración. Si el servicio vivirá siempre en un servidor Node tradicional y depende de ese ecosistema, Express sigue siendo una elección defendible.

Idea central

El análisis de viabilidad técnica conecta requisitos con capacidades y declara los trade-offs. La conclusión correcta depende del escenario: no existe el mejor framework, existe el mejor framework PARA un conjunto de requerimientos.

7. Actividades sugeridas

- Tomar un componente de clase con estado (de cualquier tutorial antiguo) y reescribirlo con hooks; explicar qué regla de los hooks rompería si el useState quedara dentro de un if.
- Construir un formulario controlado con validación y reescribirlo con una Action y useState; comparar cuánto código de estados manuales desaparece.
- Tomar una API hecha con Express en el curso y portarla a Hono; documentar qué cambió (req/res → Context, middlewares con await next(), onError) y qué quedó igual.
- Medir con curl o el navegador la diferencia de latencia entre un servicio desplegado en una sola región y uno desplegado en el edge.
- Elegir un proyecto propio y redactar media página de análisis de viabilidad: requisitos dominantes, capacidades de React/Hono que los atienden y dos trade-offs honestos.

8. Cierre

React y Hono ilustran la misma transición de fondo en el desarrollo Full Stack: de manipular mecanismos (el DOM, los objetos req/res de un runtime) a describir resultados sobre contratos estables (el estado, los Web Standards). React 19 lleva ese principio a las operaciones asíncronas de la interfaz; Hono lo lleva a la portabilidad del servicio. Entender el principio — no memorizar las APIs — es lo que permite evaluar la siguiente tecnología que aparezca.

Lecturas: Capítulo 7 (React: componentes, hooks y formularios) y Capítulo 5 (Seguridad de APIs y JWT) del Módulo III; documentación oficial en react.dev y hono.dev.